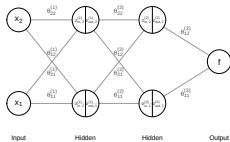
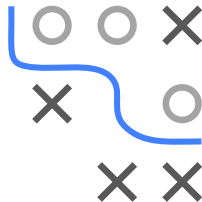


Optimization in Machine Learning

First order methods

Backpropagation and autodiff



Learning goals

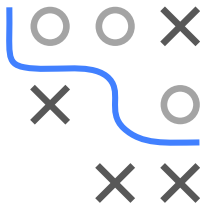
- DNNs as computational graphs
- Backpropagation via the chain rule
- Automatic differentiation and vector-Jacobian products
- Why a single backward pass suffices

COMPUTING GRADIENTS FOR DNNs

- We now train **deep neural networks (DNNs)**: models with a very large parameter vector θ
- All methods discussed in this chapter rely on the gradient of the empirical risk w.r.t. θ (SGD: on mini-batch estimates of it)

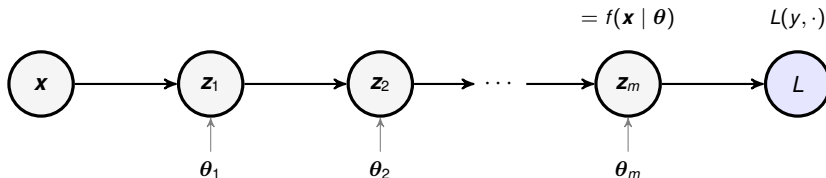
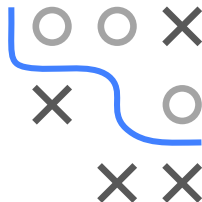
$$\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta) \in \mathbb{R}^{1 \times p}, \quad p = \dim(\theta) \text{ (millions for DNNs)}$$

- The objective is **scalar**, so this is a single row gradient – but p is huge
- **Question:** How to compute it **efficiently**?



DNNs ARE COMPUTATIONAL GRAPHS

- A DNN is a **directed acyclic graph** (DAG) of simple operations
- Nodes $\mathbf{z}_i$ hold intermediate values, edges are operations
- Layer i maps $\mathbf{z}_{i-1}$ to $\mathbf{z}_i = h_i(\mathbf{z}_{i-1}; \theta_i)$, e.g. $\mathbf{z}_i = \sigma(\mathbf{W}_i \mathbf{z}_{i-1} + \mathbf{b}_i)$



STEP 1: THE FORWARD PASS

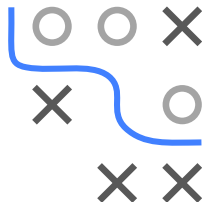
- To compute the gradient, we must first evaluate the network: the **forward pass**
- Set $\mathbf{z}_0 = \mathbf{x}$ and walk forward through the graph

$$\mathbf{z}_i = h_i(\mathbf{z}_{i-1}; \theta_i) \quad \text{for } i = 1, \dots, m, \quad L = L(y, \mathbf{z}_m)$$

- Each node value is computed once and **cached**
- These cached values are reused when computing derivatives

⇒ After the forward pass we know all $\mathbf{z}_i$ and the loss L

Caching every $\mathbf{z}_i$ is the main memory cost of what follows



STEP 2: GRADIENT VIA THE CHAIN RULE

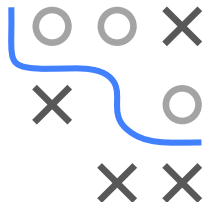
- Gradient w.r.t. the parameters θ_i of layer i factorizes:

$$\nabla_{\theta_i} L = \underbrace{\frac{\partial L}{\partial \mathbf{z}_m} \frac{\partial \mathbf{z}_m}{\partial \mathbf{z}_{m-1}} \dots \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i}}_{\text{from output back to node } i} \frac{\partial \mathbf{z}_i}{\partial \theta_i}$$

- Each factor is an elementary **Jacobian** of one operation

$$\mathbf{J}_i := \frac{\partial \mathbf{z}_i}{\partial \mathbf{z}_{i-1}} \in \mathbb{R}^{p_i \times p_{i-1}}$$

- A complicated gradient becomes a **product of simple, local derivatives**



STEP 3: BACKPROPAGATION

- The prefix under the brace is the **node adjoint**

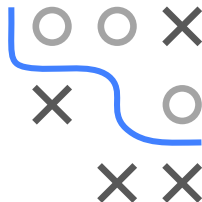
$$\bar{\mathbf{z}}_i := \nabla_{\mathbf{z}_i} L = \frac{\partial L}{\partial \mathbf{z}_i} \in \mathbb{R}^{1 \times p_i}$$

- It is **shared** by every parameter feeding into node i – compute it **once**
- Obtain all adjoints in a single **backward** sweep (reverse order):

$$\bar{\mathbf{z}}_m = \frac{\partial L}{\partial \mathbf{z}_m} \quad ; \quad \bar{\mathbf{z}}_{i-1} = \bar{\mathbf{z}}_i \mathbf{J}_i$$

- Read off each parameter gradient as $\nabla_{\theta_i} L = \bar{\mathbf{z}}_i \frac{\partial \mathbf{z}_i}{\partial \theta_i}$

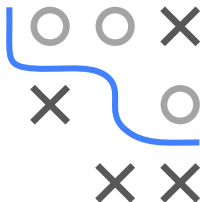
⇒ **One forward and one backward pass** give the full gradient (seed $\frac{\partial L}{\partial L} = 1$)



BACKPROPAGATION IS A SEQUENCE OF VJPS

- Every backward step $\bar{\mathbf{z}}_{i-1} = \bar{\mathbf{z}}_i \mathbf{J}_i$ is a **vector-Jacobian product** (VJP)
- Row vector times Jacobian gives a row vector again

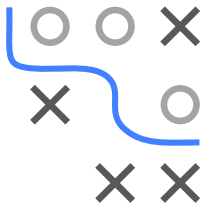
$$\begin{array}{c} \boxed{\bar{\mathbf{z}}_{i-1}} \\ 1 \times p_{i-1} \end{array} = \begin{array}{c} \boxed{\bar{\mathbf{z}}_i} \\ 1 \times p_i \end{array} \begin{array}{c} \boxed{\mathbf{J}_i} \\ p_i \times p_{i-1} \end{array}$$



- Because the loss is **scalar**, a single backward pass yields the whole gradient

AUTOMATIC DIFFERENTIATION (AUTODIFF)

- Backprop still needs manual graph building and Jacobian derivation
- **Autodiff** automates exactly this and powers modern frameworks: PyTorch (autograd), JAX, TensorFlow
- It works in two steps:
 - **Trace** the code into a graph of **primitive operations** ($+$, $\cdot$, σ , $\exp$, ...)
 - Each primitive carries a pre-registered **VJP rule**, applied in the backward pass
- The user just writes the forward computation – gradients come for free



VJP RULES AVOID THE JACOBIAN

- For a primitive $\mathbf{u} = g(\mathbf{z})$, autodiff stores a rule returning the VJP

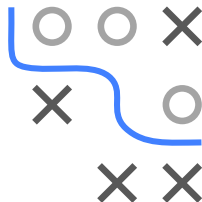
$$\text{vjp}_g(\mathbf{v}, \mathbf{z}, \mathbf{u}) = \mathbf{v} \mathbf{J}_z(g), \quad \text{without ever forming } \mathbf{J}_z(g)$$

- Example** (elementwise exp): $\mathbf{u} = \exp(\mathbf{z})$, so
 $\mathbf{J} = \text{diag}(\exp(\mathbf{z})) = \text{diag}(\mathbf{u})$

$$\mathbf{v} \mathbf{J} = \mathbf{v} \text{diag}(\mathbf{u}) = \mathbf{v} \odot \mathbf{u}$$

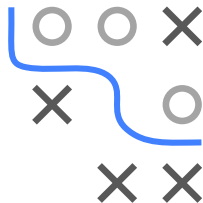
i.e. `def_vjp(exp, lambda v, u: v * u)` – an elementwise product

- The rule maps a vector to a vector; the matrix $\mathbf{J}$ is never materialized



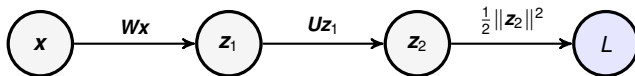
WHY THIS IS EFFICIENT

- **Naive:** build $\mathbf{J}_i \in \mathbb{R}^{p_i \times p_{i-1}}$, then multiply
 - Costs $\mathcal{O}(p_i p_{i-1})$ time and memory just to store the Jacobian
 - **VJP rule:** matrix-free, e.g. the exp rule is $\mathcal{O}(p_i)$ time and memory
 - Results stay vectors and adjoints are **recycled** across the backward pass
 - **Memory:** the forward pass must cache all node values $\mathbf{z}_i$ (activations)
- ⇒ Gradient of a scalar loss costs about **one forward and one backward pass**



WORKED EXAMPLE

- Small graph with squared output, $\theta = (W, U)$:



- **Forward:** $z_1 = Wx$, $z_2 = Uz_1$, $L = \frac{1}{2}\|z_2\|^2$
- **Backward (VJPs):** $\bar{z}_2 = z_2^\top$, $\bar{z}_1 = \bar{z}_2 U$, $\nabla_W L = \bar{z}_1 \frac{\partial z_1}{\partial W}$
- **Outlook:** the same idea computes Jacobians and feeds second order methods (Chapter 5); in code this is just `loss.backward()`

